

Jared Medlin

D209 – Data Mining I

Task 1

9-27-2024

Western Governors University

Part I – Research Question

A1. Question

Which factors are most indicative of churn?

I will be using k-nearest neighbor.

A2. Objective and Goals

The goal of this analysis is to determine which variables are indicators or predictors of customer churn. This will help the business to determine where further resources need to be placed for further research into the reasons for customer churn.

Part II – Method Justification

B1. Explanation of Classification Method

(The following was heavily sourced from: <https://www.ibm.com/topics/knn>)

K-Nearest Neighbors algorithm is a supervised learning classifier which uses proximity to make classifications or predictions about the grouping of an individual data point. The class label can be considered as the mode of the k training labels on a “plurality vote” basis.

“k” here would be the number of nearest-neighbors.

The output would be expected to be the target variable’s relation to its nearest k-neighbors and an accuracy summary.

B2. Assumption

The KNN model assumes that similar data points are close to one another, and more specifically that the nearest neighbors to the data point will be similar to it.

B3. Justification of Packages

I will be using python due to its flexibility and extensive documentation.

NumPy is a standard for using math, which I will be using.

Pandas is critical for utilizing datasets.

Matplotlib is for visualizations/graphs.

Seaborn is also for visualizations/graphs.

Sklearn's packages are typically more specialized, in this case I'll be using it for the modelling.

Part III – Data Preparation

C1. Goal

I'll need to clean and transform the data for practical use in the KNN classification model. This means that variables with string values will need to be re-expressed in numeric terms, and everything will need to be normalized.

C2. Variables

The target variable is 'Churn' which is categorical.

The other variables are:

Income – Numeric

Bandwidth_GB_Year – Numeric

Contacts – Numeric

Yearly_equip_failure – Numeric

InternetService – Categorical

Outage_sec_perweek – Numeric

Techie – Categorical

OnlineSecurity – Categorical

Port_modem – Categorical

StreamingMovies – Categorical

StreamingTV – Categorical

C3. Cleaning Steps

I'll only be cleaning the variables that I'm actually using, so variables like zip code will be left alone.

I'll start by checking for null values.

```
df.isnull().sum()  
#checking for null values and displaying as a sum
```

CaseOrder	0
Customer_id	0
Interaction	0
UID	0
City	0
State	0
County	0
Zip	0
Lat	0
Lng	0
Population	0
Area	0
TimeZone	0
Job	0
Children	0
Age	0
Income	0
Marital	0
Gender	0
Churn	0
Outage_sec_perweek	0
Email	0
Contacts	0
Yearly equip_failure	0
Techie	0
Contract	0
Port_modem	0
Tablet	0
InternetService	2129
Phone	0
Multiple	0
OnlineSecurity	0
OnlineBackup	0

The only value that comes back with nulls is InternetService. But, just like in 208, this isn't actually true, it's just reading 'None' as null, so I'll need to fix that.

```
#InternetService doesn't actually have null values, "None" is a valid value  
df['InternetService'] = df['InternetService'].fillna('NoInternet')
```

Next I'll check for duplicates:

```
df.duplicated()
#Duplicates check
```

```
0      False
1      False
2      False
3      False
4      False
...
9995   False
9996   False
9997   False
9998   False
9999   False
Length: 10000, dtype: bool
```

There are none.

Next I will create dummy variables of 1/0 for the binary categorical variables:

```
#Converting all of the categorical variables to dummy variables for the model
#Variables to be converted into new dummy columns: Churn, InternetService, Techie, OnlineSecurity, Port_modem, StreamingMovies, StreamingTV
df['DummyChurn'] = [1 if v == 'Yes' else 0 for v in df['Churn']]
df['DummyInternetService'] = [1 if v == 'Fiber Optic' else 0 for v in df['InternetService']]
df['DummyTechie'] = [1 if v == 'Yes' else 0 for v in df['Techie']]
df['DummyOnlineSecurity'] = [1 if v == 'Yes' else 0 for v in df['OnlineSecurity']]
df['DummyPort_modem'] = [1 if v == 'Yes' else 0 for v in df['Port_modem']]
df['DummyStreamingMovies'] = [1 if v == 'Yes' else 0 for v in df['StreamingMovies']]
df['DummyStreamingTV'] = [1 if v == 'Yes' else 0 for v in df['StreamingTV']]
```

Then I'll create a new dataframe with only the relevant variables:

```
#Creating a new dataframe with only relevant columns
df2 = df[['Outage_sec_perweek', 'Income', 'Contacts', 'Yearly equip_failure', 'DummyInternetService', 'Bandwidth_GB_Year', 'DummyTechie', 'DummyOnlineSecurity', 'DummyPort_modem', 'DummyStreamingMovies', 'DummyStreamingTV']]
df2.describe()
```

	Outage_sec_perweek	Income	Contacts	Yearly equip_failure	DummyInternetService	Bandwidth_GB_Year	DummyTechie	DummyOnlineSecurity	DummyPort_modem	DummyStreamingMovies	DummyStreamingTV
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	10.001848	39806.926771	0.994200	0.398000	0.440800	3392.341550	0.167900	0.357600	0.483400	0.489000	0.491000
std	2.976019	28199.916702	0.988466	0.635953	0.496508	2185.294852	0.373796	0.479317	0.499749	0.499904	0.491000
min	0.099747	348.670000	0.000000	0.000000	0.000000	155.506715	0.000000	0.000000	0.000000	0.000000	0.000000
25%	8.018214	19224.717500	0.000000	0.000000	0.000000	1236.470827	0.000000	0.000000	0.000000	0.000000	0.000000
50%	10.018560	33170.605000	1.000000	0.000000	0.000000	3279.536903	0.000000	0.000000	0.000000	0.000000	0.000000
75%	11.969485	53246.170000	2.000000	1.000000	1.000000	5586.141370	0.000000	1.000000	1.000000	1.000000	1.000000
max	21.207230	258900.700000	7.000000	6.000000	1.000000	7158.981530	1.000000	1.000000	1.000000	1.000000	1.000000

Then I need to normalize the new set from 0 to 1:

```
#Normalizing
df3 = pd.DataFrame(preprocessing.MinMaxScaler().fit_transform(df2), columns=df2.columns)
df3.describe()
```

	Outage_sec_perweek	Income	Contacts	Yearly equip_failure	DummyInternetServi
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	0.469128	0.152612	0.142029	0.066333	0.44081
std	0.140994	0.109069	0.141209	0.105992	0.49651
min	0.000000	0.000000	0.000000	0.000000	0.00000
25%	0.375150	0.073007	0.000000	0.000000	0.00000
50%	0.469919	0.126945	0.142857	0.000000	0.00000
75%	0.562347	0.204591	0.285714	0.166667	1.00000
max	1.000000	1.000000	1.000000	1.000000	1.00000

Data is now ready to be used.

C4. Cleaned Data Set

SEE ATTACHED FILE

Part IV – Analysis

D1. Splitting Data

SEE ATTACHED .CSV FILES

D2. Description of Analysis Technique

For a knn model you must, of course, select a k-value. This is the number of neighboring points that will be considered “nearest”. It’s perfectly viable to select a random k-value and try and find one that gives you a good accuracy number, but by using GridSearchCV it’s possible to find the *best* k-value. It will try the values in the given range and select the best one automatically.

Source for code: https://scikit-learn.org/dev/modules/generated/sklearn.model_selection.GridSearchCV.html

```
#Selecting for best k code was from https://scikit-learn.

#Selecting value for k using GridSearchCV
paramgrid = {'n_neighbors' : np.arange(1, 33)}

#Defining knn
knn = KNeighborsClassifier()

#Defining knn2 as GridSearchCV
gridknn = GridSearchCV(knn, paramgrid, cv=5)

#Fitting to training data
gridknn.fit(xtrain, ytrain)

#Finding best k
gridknn.best_params_

{'n_neighbors': 14}
```

```
#High score
gridknn.best_score_
```

```
0.8305
```

The best k-value in the given range is 14.

Creating the model is fairly straightforward, but I used <https://www.ibm.com/topics/knn> for help.

```
#Now I'll do the initial model
knn = KNeighborsClassifier(n_neighbors = 14)
knn.fit(xtrain, ytrain)
```

```
#For prediction outcomes
ypredict = knn.predict(xtest)
```

```
#Model accuracy
print('Training Accuracy: ', knn.score(xtrain, ytrain))
print('Testing Accuracy: ', knn.score(xtest, ytest))
```

```
Training Accuracy: 0.850625
Testing Accuracy: 0.8225
```

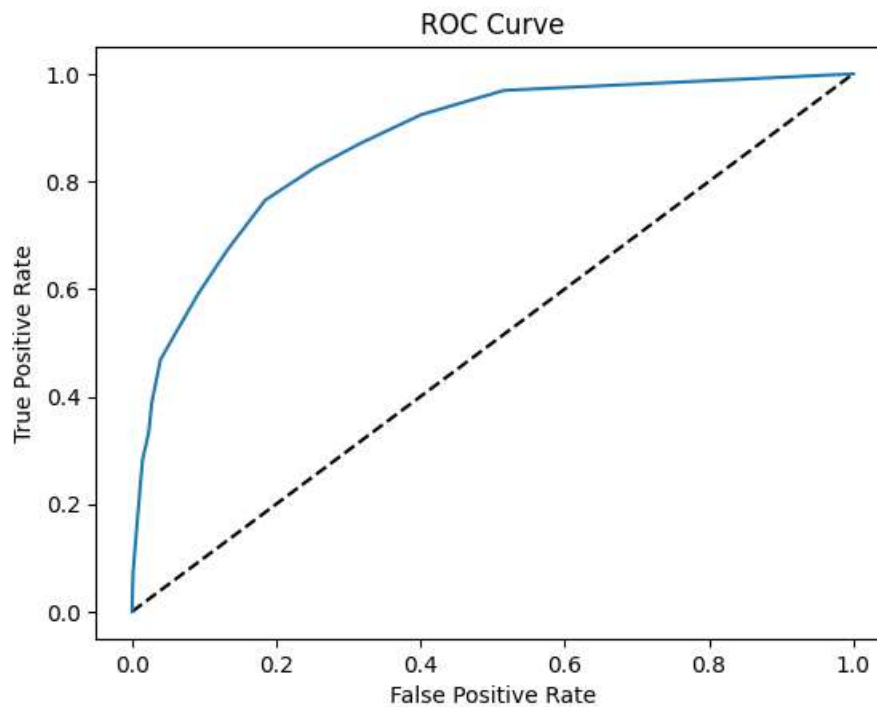
To get the area under the curve score I used code from scikit's website:

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_curve.html

```
#AUC Score stuff is from scikit's website, such as: https
yprob = knn.predict_proba(xtest)[: , 1]
fpr, tpr, thresholds = roc_curve(ytest, yprob)
plt.plot([0, 1], [0, 1], 'k--')
plt.plot(fpr, tpr)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.show()
print('AUC score: ', {roc_auc_score(ytest, yprob)})
print(classification_report(ytest, ypredict))
```

And the output:



```
AUC score: {0.8722011369785463}
precision  recall  f1-score  support
0.0        0.84    0.93    0.88    1447
1.0        0.75    0.53    0.62    553

accuracy          0.82    2000
macro avg         0.80    0.73    0.75    2000
weighted avg      0.82    0.82    0.81    2000
```


D3. Code

SEE CODE ATTACHED

Part V – Summary and Implications

E1. Explanation

The accuracy of the model was 0.851, and the area under the curve was 0.872.

The accuracy of the model is somewhat self-explanatory, it's how accurate the model is at making predictions relative to the total number of predictions made (in terms of a confusion matrix). So in this case the model made a correct prediction 85.1% of the time, which is good.

Area under the curve is as graphed; the true positive rate over the false positive rate. A model that is completely randomly guessing is represented by the dotted line. A random model is correct 50% of the time, so if you have more area under your curve than 50% of the area your model is outperforming baseline. This model has 87.2% of the area under the curve, meaning it is far outperforming baseline, and is thus a good model.

E2. Results and Implications

Again, the accuracy of the model was 0.851 and the area under the curve was 0.872.

The model's accuracy and AUC score being so high is a good sign. This means that the model can correctly guess whether a customer will churn roughly 85% of the time with the given variables. This could be a powerful tool for the business to predict levels of churn in advance given data.

E3. Limitations

The model is limited to the variables given for its predictive ability. What I mean is, while the model is good at predicting churn based on static data from a limited list of variables, it would be unwise to wheel this out and start using it as a predicting machine for how much churn you will have because there are many more variables the model doesn't account for.

E4. Course of Action

Given that the model is fairly accurate, it would be wise to train it by adding more and more variables relevant to the business into it in order to discern how useful it could actually be before actually utilizing it. As I discussed before, the model as it is now is fairly limited in practical use. By expanding the dataset to include other outside variables that can effect churn, the model could eventually prove useful for predicting the amount of churn that will occur in a fiscal period.

Part VI: Demonstration

F. Panopto Video

SEE ATTACHED LINK

G. Sources

IBM. (2022, August 23). *Learn classification algorithms using Python and scikit-learn*. IBM Developer. <https://developer.ibm.com/tutorials/learn-classification-algorithms-using-python-and-scikit-learn/>

IBM. (n.d.). *What is k-nearest neighbors (k-NN)?* IBM. <https://www.ibm.com/topics/knn>

scikit-learn developers. (n.d.). *sklearn.metrics.roc_auc_score* — *scikit-learn 1.3.0 documentation*. Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html

scikit-learn developers. (n.d.). *sklearn.model_selection.GridSearchCV* — *scikit-learn 1.4.0 documentation*. Scikit-learn. https://scikit-learn.org/dev/modules/generated/sklearn.model_selection.GridSearchCV.html

scikit-learn developers. (n.d.). *sklearn.neighbors.KNeighborsClassifier* — *scikit-learn 1.3.0 documentation*. Scikit-learn. <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>

H. Sources Cited

Sources cited in text or code where applicable.

I. Demonstrate Professionalism

Thank you for reading!